



UNIVERSIDAD DE LA FRONTERA

Desarrollo de aplicación cliente-servidor con uso de sockets

Chat cliente-servidor sobre sockets TCP (SOCK_STREAM)

Taller de Redes

Integrantes:

Luis Bravo

Felipe Delgado

Raúl Manríquez

Docente: Flavio Rojas

Fecha de entrega: 31-08-2026

1. Introducción

La comunicación entre procesos a través de una red es un concepto clave en el desarrollo de sistemas distribuidos. Para lograr esta interacción existe un mecanismo llamado socket, el cual permite establecer un canal de comunicación e intercambio de datos entre dos procesos, ya sea dentro del mismo equipo o entre equipos distintos conectados a una red.

En el marco de esta actividad del Taller de Redes, se implementó una aplicación basada en el modelo cliente-servidor mediante el uso de sockets. Con este trabajo se buscó comprender de forma práctica el funcionamiento de los sockets, su configuración en código y los métodos principales necesarios para establecer, mantener y cerrar correctamente una comunicación en red.

El objetivo general de este informe es documentar el desarrollo de la aplicación construida utilizando el protocolo de transporte TCP, aplicando los mecanismos requeridos para abrir la conexión, transmitir información, validar las respuestas del servidor y cerrar el canal de comunicación de forma ordenada.

Para la implementación se eligió el protocolo TCP (Transmission Control Protocol), utilizando sockets de tipo SOCK_STREAM. Se optó por TCP debido a que es un protocolo orientado a la conexión que garantiza la entrega ordenada y confiable de los datos, característica necesaria para el tipo de aplicación desarrollada.

La aplicación desarrollada consiste en un chat cliente-servidor básico programado en Python. El cliente se conecta al servidor a través de la interfaz de loopback, utilizando la dirección IP 127.0.0.1 y el puerto 5050. Una vez establecida la sesión, el usuario puede interactuar mediante mensajes de texto libres y también mediante comandos específicos: /hora para obtener la hora actual del servidor, /archivo <nombre> para solicitar la lectura de un archivo de texto almacenado en el servidor, y /salir para finalizar la conexión de forma ordenada.

Finalmente, el programa incorpora validaciones básicas sobre los mensajes recibidos, manejo de excepciones ante distintos escenarios de falla en la comunicación, y un cierre adecuado de los sockets al finalizar la sesión, lo que permite analizar en la práctica los distintos estados por los que atraviesa una conexión TCP durante su ejecución.

2. Marco conceptual

2.1 ¿Qué es un socket?

Un socket es un mecanismo que permite establecer comunicación entre dos procesos, posibilitando el envío y la recepción de datos a través de una red. En una aplicación cliente-servidor, los sockets constituyen el canal de comunicación entre ambos componentes: cada extremo de la comunicación (cliente y servidor) posee su propio socket.

Un socket se crea utilizando la función `socket()`. En la aplicación desarrollada, tanto el cliente como el servidor lo crean de la siguiente forma:

```
cliente_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

2.2 Modelo cliente-servidor

El modelo cliente-servidor consiste en una arquitectura en la cual dos componentes cumplen distintas funciones dentro de la comunicación. El servidor es la parte que permanece a la espera de conexiones de los clientes: dentro de la aplicación, el servidor crea un socket, le asocia una dirección IP y un puerto, y queda a la espera de conexiones por medio de `listen()` y `accept()`.

El cliente, en cambio, es quien inicia la comunicación conectándose al servidor por medio de `connect()`. Una vez establecida la conexión, ambos extremos pueden intercambiar mensajes mediante `send()` y `recv()`.

En términos generales, el flujo de la aplicación consiste en que el cliente se conecta al servidor, le envía un mensaje, el servidor lo recibe y lo procesa, y finalmente responde al cliente.

2.3 Protocolo TCP

TCP (Transmission Control Protocol) es un protocolo de transporte orientado a la conexión: antes de intercambiar datos, el cliente y el servidor deben establecer primero una conexión. En la aplicación desarrollada, TCP se utiliza mediante un socket de tipo `SOCK_STREAM`; el cliente usa `connect()` para conectarse al servidor, mientras que el servidor usa `listen()` y `accept()` para aceptar conexiones entrantes. Una vez establecida la conexión, TCP garantiza que los datos se reciban en el mismo orden en que fueron enviados y retransmite automáticamente los segmentos que se pierdan en el camino.

2.4 Protocolo UDP

UDP (User Datagram Protocol) es un protocolo de transporte que no requiere establecer una conexión persistente entre servidor y cliente antes de enviar datos: cada datagrama se envía de forma independiente. En esta aplicación no se utiliza UDP, pero se incluye su descripción a modo de comparación con TCP. Se seleccionó TCP para este trabajo debido a que se buscaba mantener una comunicación persistente entre cliente y servidor durante toda la sesión, permitiendo un intercambio de mensajes orientado a la conexión, con la fiabilidad que ese modelo ofrece.

2.5 Comparación entre TCP y UDP

Característica	TCP	UDP
Orientación	Orientado a la conexión	No orientado a la conexión
Confiabilidad	Garantiza entrega y orden	No garantiza entrega ni orden
Retransmisión	Automática ante pérdida	No existe
Establecimiento de conexión	Three-way handshake (SYN / SYN-ACK / ACK)	No requiere negociación previa
Tipo de socket en Python	<code>SOCK_STREAM</code>	<code>SOCK_DGRAM</code>

Característica	TCP	UDP
Velocidad / overhead	Mayor overhead, más lento	Menor overhead, más rápido
Casos de uso típicos	Chat, transferencia de archivos, HTTP	Streaming, VoIP, juegos en tiempo real, DNS
Uso en esta aplicación	Utilizado (protocolo elegido)	No utilizado (solo referencial)

2.6 Dirección IP y puerto

La dirección IP permite identificar el destino de una comunicación dentro de una red. En esta aplicación, la IP utilizada es 127.0.0.1, dirección que corresponde a la interfaz de loopback y que permite que la conexión entre cliente y servidor se realice dentro del mismo equipo, sin salir a una red física. El puerto utilizado es el 5050, número que permite identificar, dentro de ese equipo, el servicio específico con el cual se desea establecer la conexión.

2.7 Familia de direcciones y tipo de socket

El socket que utiliza la aplicación se crea de la siguiente forma:

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

El parámetro AF_INET indica que se utilizará direccionamiento IPv4, mientras que el parámetro SOCK_STREAM indica que se utilizará un socket de flujo orientado a la conexión (propio de TCP), que permite enviar y recibir datos de forma confiable entre los dos dispositivos.

3. Diseño de la solución

3.1 Descripción de la aplicación

La aplicación desarrollada es un sistema básico de comunicación cliente-servidor implementado en Python mediante el uso de sockets TCP. Permite el intercambio de mensajes entre cliente y servidor a través de una conexión establecida mediante la dirección IP 127.0.0.1 y el puerto 5050. El servidor permanece a la espera de conexiones, lee los mensajes que el cliente le envía y responde a cada uno de ellos según corresponda.

3.2 Funcionalidades implementadas

- Conexión cliente-servidor mediante sockets TCP.
- Envío y recepción de mensajes de texto libres, con respuesta tipo eco por parte del servidor.
- Comando /hora: consulta la hora actual del servidor.
- Comando /archivo <nombre>: solicita el contenido de un archivo de texto almacenado en el servidor, con protección contra accesos fuera del directorio permitido (Path Traversal).
- Comando /salir: finaliza la comunicación de forma ordenada, con confirmación del servidor.

- Validación de mensajes vacíos, tanto en el cliente como en el servidor.
- Manejo de errores de red: servidor no disponible, puerto en uso, desconexión abrupta del cliente, y agotamiento de tiempo de espera (timeout).

3.3 Arquitectura cliente-servidor

Se utiliza una arquitectura compuesta por un programa servidor y un programa cliente. El servidor es quien crea el socket TCP y lo vincula a la IP y puerto definidos, permaneciendo a la espera de conexiones y de mensajes entrantes. El cliente es quien inicia la comunicación estableciendo la conexión con el servidor; una vez conectados, el cliente envía mensajes y recibe las respuestas correspondientes desde el servidor.

3.4 Flujo de comunicación

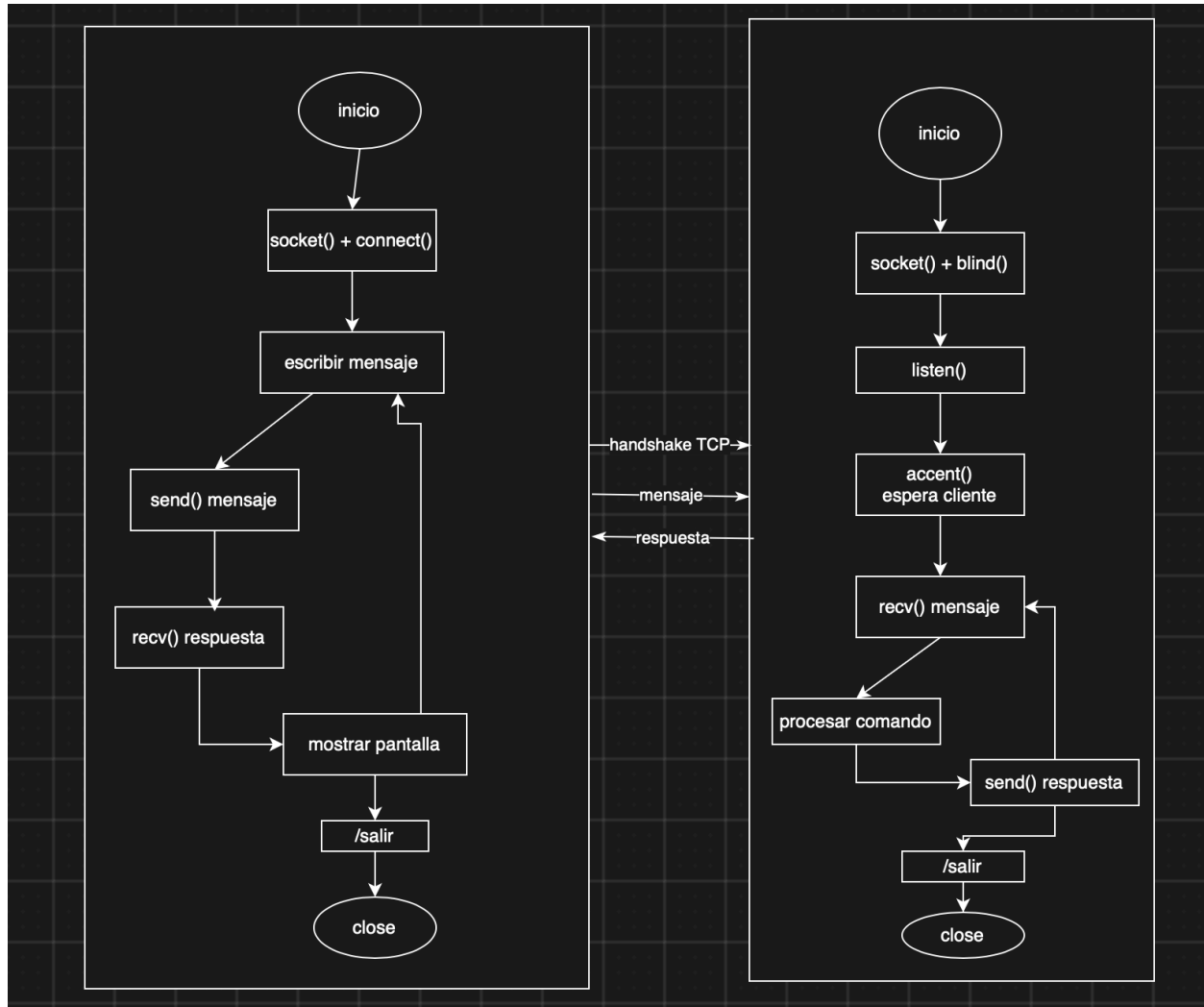
- Se inicia el servidor.
- El servidor crea un socket TCP utilizando AF_INET y SOCK_STREAM.
- El servidor asocia el socket a la dirección 127.0.0.1 y al puerto 5050 mediante bind().
- El servidor comienza a escuchar conexiones entrantes mediante listen().
- Se inicia el cliente y se crea su propio socket TCP.
- El cliente intenta establecer una conexión con el servidor mediante connect().
- Una vez establecida la conexión, el cliente solicita al usuario un mensaje.
- El mensaje ingresado es validado para evitar el envío de una entrada vacía.
- El cliente envía el mensaje al servidor mediante send().
- El servidor recibe el mensaje mediante recv() y lo procesa.
- El servidor genera una respuesta según el contenido del mensaje (texto libre o comando).
- El servidor envía la respuesta al cliente mediante send().
- El cliente recibe la respuesta mediante recv() y la muestra en pantalla.
- Los pasos de envío y recepción se repiten mientras la sesión permanezca activa, hasta que se recibe el comando /salir o se produce un error de conexión.

3.5 Formato de los mensajes intercambiados

Los mensajes se transmiten como texto plano codificado en UTF-8. El cliente envía el texto ingresado por el usuario (previamente validado), y el servidor responde con una cadena de texto que incluye un prefijo identificador según el tipo de respuesta —por ejemplo [SERVIDOR], [ARCHIVO: nombre] o [ERROR]— lo que facilita distinguir en pantalla el tipo de resultado obtenido.

3.6 Diagrama de actividad

El siguiente diagrama representa el flujo completo de la aplicación, diferenciando mediante carriles el comportamiento del servidor (izquierda) y del cliente (derecha). Cada actividad indica, cuando corresponde, el método de socket asociado a dicho paso.



3.7 Decisiones de diseño relevantes

Durante el desarrollo se tomaron las siguientes decisiones de diseño, orientadas a la simplicidad, la robustez frente a errores y la facilidad de pruebas:

- Se optó por un servidor de tipo iterativo (atiende un cliente a la vez) en lugar de uno concurrente, ya que el objetivo de la actividad es demostrar el uso correcto de los métodos de socket y no la concurrencia; se documenta como posible mejora futura en las conclusiones.
- Se utilizó `setsockopt()` con la opción `SO_REUSEADDR` para permitir reiniciar el servidor rápidamente durante las pruebas, evitando quedar bloqueado por el estado `TIME_WAIT` que deja TCP tras cerrar una conexión.
- Se definió un timeout de 15 segundos en el cliente mediante `settimeout()`, de modo que la aplicación no quede esperando indefinidamente si el servidor deja de responder.
- Se incorporó una validación de tipo Path Traversal en el comando `/archivo`: el servidor compara el nombre base del archivo solicitado contra la ruta resuelta, de forma que no sea posible solicitar archivos fuera del directorio `server/archivos/` (por ejemplo, intentando `/archivo ../server.py`).

- Se decidió transmitir los mensajes como texto plano en UTF-8, sin un formato binario adicional, dado que el volumen y tipo de datos intercambiados (comandos y texto corto) no lo requería para los fines de esta actividad.

4. Desarrollo e implementación

4.1 Lenguaje y herramientas

La aplicación fue desarrollada en Python 3 (versión 3.9 o superior), utilizando exclusivamente el módulo estándar socket para la comunicación en red, junto con los módulos datetime (comando /hora), os (manejo del directorio y rutas de archivos) y errno (identificación explícita del error de puerto en uso). No se utilizaron bibliotecas externas ni frameworks de red de alto nivel, con el fin de trabajar directamente sobre la API de sockets provista por el sistema operativo.

4.2 Configuración de IP y puerto

Tanto el servidor como el cliente definen como constantes la dirección HOST = "127.0.0.1" y el puerto PORT = 5050. El tamaño del buffer de lectura se fija en BUFFER_SIZE = 1024 bytes.

4.3 Funcionamiento del servidor

El servidor (server.py) sigue la secuencia clásica de un servidor TCP: crea el socket con socket(), configura la reutilización de dirección con setsockopt(), lo asocia a la dirección y puerto con bind(), habilita la escucha con listen(5) y entra en un bucle que acepta conexiones con accept(). Cada cliente aceptado es atendido por la función atender_cliente(), que ejecuta un bucle de recepción (recv()) y envío (send()) de mensajes hasta que el cliente cierra la conexión, se desconecta abruptamente, o solicita /salir. La función procesar_mensaje() concentra la lógica de validación e interpretación de los comandos disponibles.

```
import socket
import datetime
import os
import sys
import errno

# =====
# CONFIGURACIÓN DEL SERVIDOR
# =====
HOST = "127.0.0.1" # Loopback / Localhost
PORT = 5050        # Puerto de escucha del servidor
BUFFER_SIZE = 1024 # Tamaño máximo de buffer de lectura en bytes

# Directorio de archivos para la funcionalidad /archivo <nombre>
ARCHIVOS_DIR = os.path.join(os.path.dirname(__file__), "archivos")
os.makedirs(ARCHIVOS_DIR, exist_ok=True)

def procesar_mensaje(mensaje: str) -> str:
    """
    Valida el mensaje recibido y genera la respuesta correspondiente.
    Comandos disponibles:
    - /hora: Retorna la hora actual del servidor.
```

```

- /archivo <nombre>: Retorna el contenido de un archivo de texto.
- /salir: Solicita la desconexión del cliente.
Maneja excepciones de I/O de archivos de manera segura.
"""
mensaje = mensaje.strip()

# Validación de mensaje vacío
if not mensaje:
    return "[ERROR] Mensaje vacío no permitido."

# Comando de salida
if mensaje == "/salir":
    return "__CERRAR__"

# Comando de hora actual
if mensaje == "/hora":
    ahora = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return f"[SERVIDOR] Hora actual: {ahora}"

# Comando de lectura de archivo
if mensaje.startswith("/archivo"):
    partes = mensaje.split(maxsplit=1)
    if len(partes) < 2:
        return "[ERROR] Uso correcto: /archivo <nombre_archivo.txt>"

    nombre_archivo = partes[1].strip()
    ruta = os.path.join(ARCHIVOS_DIR, nombre_archivo)

    # Prevención básica de Path Traversal (evita accesos tipo ../../)
    if os.path.basename(ruta) != nombre_archivo:
        return "[ERROR] Nombre de archivo no válido por motivos de seguridad."

    try:
        with open(ruta, "r", encoding="utf-8") as f:
            contenido = f.read()
            return f"[ARCHIVO: {nombre_archivo}]\n{contenido}"
    except FileNotFoundError:
        return f"[ERROR] El archivo '{nombre_archivo}' no existe en el servidor."
    except PermissionError:
        return f"[ERROR] Permisos insuficientes para leer '{nombre_archivo}'."
    except UnicodeDecodeError:
        return f"[ERROR] El archivo '{nombre_archivo}' no es una cadena de texto UTF-8
válida."
    except OSError as e:
        return f"[ERROR] Error de lectura de archivo en servidor: {e}"

# Respuesta por defecto para mensajes comunes
return f"[SERVIDOR] Recibido: {mensaje}"

def atender_cliente(cliente_socket: socket.socket, direccion: tuple):
    """
    Atiende la comunicación interactiva con un cliente conectado.
    Maneja desconexiones limpias (0 bytes / FIN) y desconexiones forzadas
    (ConnectionResetError / RST).
    """
    print(f"[CONEXION] Cliente conectado exitosamente desde {direccion}")
    try:
        while True:
            try:
                # CASO DE ERROR 1 (SERVIDOR): Desconexión ordenada vs forzada

```



```

        # recv() se bloquea esperando datos desde la capa de transporte TCP.
        datos = cliente_socket.recv(BUFFER_SIZE)

        # CIERRE ORDENADO: cuando el cliente llama a socket.close(), TCP envía un FIN.
        # En Python socket, la recepción de FIN se manifiesta cuando recv() retorna
b"".
        if not datos:
            print(f"[INFO] El cliente {direccion} cerró la conexión ordenadamente (FIN
recibido / 0 bytes).")
            break

        mensaje = datos.decode("utf-8", errors="replace")
        print(f"[MENSAJE] {direccion} -> {mensaje}")

        # Procesar la lógica del mensaje
        respuesta = procesar_mensaje(mensaje)

        # Si el usuario solicitó /salir
        if respuesta == "__CERRAR__":
            try:
                cliente_socket.send("[SERVIDOR] Confirmando cierre de sesión...
Bye!".encode("utf-8"))
            except (ConnectionResetError, BrokenPipeError):
                pass
            print(f"[INFO] Cliente {direccion} solicitó desconexión mediante /salir.")
            break

        # Envío de respuesta al cliente a través del socket TCP
        cliente_socket.send(respuesta.encode("utf-8"))

        # CAÍDA FORZADA: el cliente cerró la aplicación bruscamente. El SO responde con
RST.
        except ConnectionResetError:
            print(f"[ADVERTENCIA] Conexión reiniciada abruptamente por el cliente
{direccion} (ConnectionResetError / RST).")
            break
        # CAÑERÍA ROTA: se intenta escribir en un socket que la otra parte ya cerró.
        except BrokenPipeError:
            print(f"[ADVERTENCIA] Cañería rota (BrokenPipeError) al intentar enviar a
{direccion}. Socket cerrado.")
            break
        # Errores generales de socket
        except socket.error as e:
            print(f"[ERROR SOCKET] Error en la comunicación TCP con cliente {direccion}:
{e}")
            break
        # Captura de seguridad para cualquier otra excepción no prevista
        except Exception as e:
            print(f"[ERROR INESPERADO] Error en procesamiento del cliente {direccion}:
{e}")
            break
    finally:
        # CASO DE ERROR 4 (SERVIDOR): cierre garantizado de socket cliente
        try:
            cliente_socket.close()
        except OSError:
            pass
        print(f"[DESCONEXION] Socket y recursos del cliente {direccion} liberados de forma
segura.\n")

```

```

def iniciar_servidor():
    """
    Inicializa el socket servidor, realiza bind(), listen() y bucle de aceptado.
    Maneja errores de puerto en uso (OSError) e interrupción por teclado (KeyboardInterrupt).
    """
    servidor_socket = None
    try:
        # socket(): Crear socket IPv4 (AF_INET) y TCP (SOCK_STREAM)
        servidor_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # SO_REUSEADDR permite reutilizar la dirección/puerto inmediatamente tras reiniciar el
        # servidor (evita quedar bloqueado por el estado TIME_WAIT del protocolo TCP).
        servidor_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        # CASO DE ERROR 2 (SERVIDOR): Puerto ya en uso (OSError)
        # bind() asocia el socket a la IP y Puerto especificados.
        servidor_socket.bind((HOST, PORT))

        # listen(): habilita el socket para recibir conexiones entrantes (cola de espera = 5)
        servidor_socket.listen(5)
        print(f"[INICIO] Servidor TCP listo y escuchando en {HOST}:{PORT}")

        # Bucle principal de aceptación de conexiones
        while True:
            print("[ESPERA] Esperando conexión entrante de algún cliente...")
            try:
                # accept() bloquea hasta completar el Handshake TCP de 3 vías con un cliente
                cliente_socket, direccion = servidor_socket.accept()
                # Delegar la atención del cliente conectado
                atender_cliente(cliente_socket, direccion)
            # CASO DE ERROR 3 (SERVIDOR): parada manual por consola (Ctrl+C)
            except KeyboardInterrupt:
                raise
            except Exception as e:
                print(f"[ERROR ACCEPT] Ocurrió un problema al aceptar la conexión: {e}")

        except OSError as e:
            # Captura específica si el puerto ya está en uso (WinError 10048 en Windows o
            # EADDRINUSE en Linux)
            if e.errno == errno.EADDRINUSE or "10048" in str(e):
                print(f"[ERROR CRÍTICO] El puerto {PORT} ya está en uso por otra aplicación
                (Address already in use).")
            else:
                print(f"[ERROR OS] Error de sistema operativo al iniciar socket del servidor:
                {e}")
        except KeyboardInterrupt:
            print("\n[APAGADO] Servidor detenido manualmente desde consola (KeyboardInterrupt /
            Ctrl+C).")
        except Exception as e:
            print(f"[ERROR CRÍTICO] Excepción no controlada en el servidor: {e}")
    finally:
        # CASO DE ERROR 4 (SERVIDOR): cierre garantizado del socket principal
        if servidor_socket:
            print("[LIMPIEZA] Cerrando el socket principal del servidor...")
            try:
                servidor_socket.close()
            except OSError as e:
                print(f"[ERROR] Error al cerrar socket principal: {e}")
        print("[SERVIDOR] Proceso del servidor finalizado correctamente.")

```

```
if __name__ == "__main__":
    iniciar_servidor()
```

4.4 Funcionamiento del cliente

El cliente (client.py) crea su socket con `socket()`, configura un tiempo de espera máximo con `settimeout()` y se conecta al servidor con `connect()`. Luego entra en un bucle interactivo: solicita al usuario un mensaje por consola, lo valida (evitando cadenas vacías), lo envía con `send()` y espera la respuesta del servidor con `recv()`, mostrándola en pantalla. El bucle se repite hasta que el usuario envía /salir o se produce un error de red no recuperable, momento en el cual el socket se cierra mediante `close()` dentro de un bloque `finally`.

```
import socket
import sys

# =====
# CONFIGURACIÓN DEL CLIENTE
# =====
HOST = "127.0.0.1"      # IP del servidor (loopback)
PORT = 5050             # Puerto del servidor
BUFFER_SIZE = 1024      # Tamaño de buffer de recepción
TIMEOUT_SEGUNDOS = 15.0 # Tiempo límite de espera para respuestas del socket (segundos)

def iniciar_cliente():
    """
    Inicia el cliente TCP, establece conexión con el servidor,
    valida la entrada del usuario, impone timeouts y maneja
    excepciones de red de forma segura con bloque finally.
    """
    cliente_socket = None
    try:
        # socket(): Crear socket IPv4 (AF_INET) y TCP (SOCK_STREAM)
        cliente_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # CASO DE ERROR 3 (CLIENTE): configuración de límite de tiempo (Timeout)
        cliente_socket.settimeout(TIMEOUT_SEGUNDOS)

        print(f"[CONECTANDO] Intentando conectar a {HOST}:{PORT}...")
        # connect(): inicia el Handshake TCP (SYN -> SYN-ACK -> ACK)
        cliente_socket.connect((HOST, PORT))

        # CASO DE ERROR 1 (CLIENTE): servidor apagado o puerto cerrado
        except ConnectionRefusedError:
            print(f"[ERROR CONEXION] No se pudo conectar a {HOST}:{PORT}. El servidor está apagado o el puerto está cerrado.")
            return
        except socket.timeout:
            print(f"[ERROR TIMEOUT] Tiempo agotado ({TIMEOUT_SEGUNDOS}s) intentando conectar con el servidor.")
            return
        except socket.gaierror:
            print(f"[ERROR RED] No se pudo resolver la dirección host '{HOST}'. Verifique la dirección IP.")
            return
        except OSError as e:
            print(f"[ERROR OS] Error de sistema operativo al intentar conectar: {e}")
            return
```

```

except Exception as e:
    print(f"[ERROR INESPERADO] Error al intentar conectar: {e}")
    return
else:
    # El bloque 'else' se ejecuta ÚNICAMENTE si el bloque try no arrojó ninguna excepción
    print(f"[CONEXION EXITOSA] Conectado con éxito al servidor en {HOST}:{PORT}")
    print("Instrucciones: Escribe un mensaje o usa los comandos: /hora, /archivo
<nombre.txt>, /salir\n")

    # Bucle interactivo de envío y recepción
    try:
        while True:
            try:
                mensaje = input("Tú: ")
            except (KeyboardInterrupt, EOFError):
                print("\n[SALIDA] Interrupción detectada en consola (Ctrl+C / Ctrl+D).
Cerrando sesión...")
                break

            # CASO DE ERROR 4 (CLIENTE): validación previa al envío
            mensaje_limpio = mensaje.strip()
            if not mensaje_limpio:
                print("[ADVERTENCIA] No se permiten mensajes vacíos. Por favor ingresa
texto.")
                continue

            try:
                # send(): envía los bytes codificados en UTF-8 a través del socket TCP
                cliente_socket.send(mensaje_limpio.encode("utf-8"))

                # CASO DE ERROR 2 Y 3 (CLIENTE): caída de conexión y Timeout en recv()
                datos = cliente_socket.recv(BUFFER_SIZE)

                # Detección de cierre ordenado por parte del servidor (FIN -> 0 bytes)
                if not datos:
                    print("[INFO SERVIDOR] El servidor cerró la conexión ordenadamente
(FIN recibido).")
                    break

                respuesta = datos.decode("utf-8", errors="replace")
                print(f"{respuesta}\n")

                if mensaje_limpio == "/salir":
                    break

            except socket.timeout:
                print(f"[ERROR TIMEOUT] El servidor tardó más de {TIMEOUT_SEGUNDOS}s en
responder.")
            except ConnectionResetError:
                print("[ERROR CONEXION] Se perdió la conexión con el servidor
(ConnectionResetError / RST).")
                break
            except BrokenPipeError:
                print("[ERROR CONEXION] Cañería rota (BrokenPipeError). El servidor cerró
el socket.")
                break
            except socket.error as e:
                print(f"[ERROR SOCKET] Ocurrió un error en la transmisión de datos: {e}")
                break
            except Exception as e:
                print(f"[ERROR INESPERADO] Excepción no controlada durante el chat: {e}")

```

```

        break
    finally:
        # CASO DE ERROR 5 (CLIENTE): cierre garantizado del socket cliente
        if cliente_socket:
            print("[LIMPIEZA] Cerrando el socket del cliente...")
            try:
                cliente_socket.close()
            except OSError as e:
                print(f"[ERROR] Error al cerrar socket cliente: {e}")
            print("[CLIENTE] Conexión y sesión finalizadas.")

if __name__ == "__main__":
    iniciar_cliente()

```

4.5 Validaciones implementadas

- Cliente y servidor: rechazo de mensajes vacíos o compuestos solo por espacios en blanco.
- Servidor: validación de la sintaxis del comando /archivo (debe incluir un nombre de archivo).
- Servidor: prevención de Path Traversal comparando el nombre base (`os.path.basename()`) de la ruta resuelta contra el nombre de archivo recibido, bloqueando intentos como /archivo ../server.py.
- Servidor: manejo explícito de `FileNotFoundError`, `PermissionError`, `UnicodeDecodeError` y `OSError` al leer archivos.

4.6 Manejo de errores

El programa distingue explícitamente distintos escenarios de error de red. En el servidor: `ConnectionResetError` (el cliente cerró abruptamente la conexión y el sistema operativo recibe un segmento RST), `BrokenPipeError` (se intenta escribir en un socket cuyo otro extremo ya cerró) y `OSError` al iniciar el servidor cuando el puerto ya está en uso (`EADDRINUSE`). En el cliente: `ConnectionRefusedError` (el servidor está apagado o el puerto está cerrado), `socket.timeout` (el servidor no respondió dentro del tiempo configurado) y `socket.gaierror` (no fue posible resolver la dirección del host). Todas estas excepciones se capturan de forma específica antes de una captura genérica de respaldo (`except Exception`), informando en cada caso un mensaje descriptivo por consola.

4.7 Cierre de conexiones y liberación de recursos

Tanto el cliente como el servidor utilizan bloques `try/finally` para garantizar que el método `close()` se invoque siempre sobre los sockets abiertos, sin importar si la sesión terminó de forma normal (mediante /salir) o por un error. En el servidor, el socket de cada cliente se cierra dentro de `atender_cliente()`, mientras que el socket principal del servidor se cierra en el bloque `finally` de `iniciar_servidor()`, lo que garantiza la liberación del puerto incluso ante una interrupción manual (`Ctrl+C`) o un error crítico no controlado.

5. Descripción de métodos de sockets

A continuación se documentan los diez métodos de sockets utilizados en el desarrollo de la aplicación. Todos los métodos aquí listados fueron efectivamente utilizados en el código fuente, tal como se indica en la referencia de cada uno.

1. socket()

Campo	Detalle
Propósito	Crea un nuevo objeto socket, es decir, un punto final de comunicación de red, especificando la familia de direcciones y el tipo de socket a utilizar.
Sintaxis (Python)	<code>socket.socket(family=AF_INET, type=SOCK_STREAM, proto=0)</code>
Parámetros principales	family: familia de direcciones (AF_INET para IPv4). type: tipo de socket (SOCK_STREAM para TCP, SOCK_DGRAM para UDP). proto: número de protocolo, normalmente 0 para que el sistema lo determine automáticamente.
Valor retornado	Un objeto socket que representa el punto final de la comunicación.
Protocolo	Ambos (TCP y UDP); el protocolo se define mediante el parámetro type.
Ejemplo de uso	<code>s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)</code>
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 154 (servidor) y client.py, línea 22 (cliente).

2. setsockopt()

Campo	Detalle
Propósito	Permite configurar opciones a nivel de socket, como el reuso de la dirección local, tamaños de buffer o timeouts a bajo nivel.
Sintaxis (Python)	<code>socket.setsockopt(level, optname, value)</code>
Parámetros principales	level: nivel al que pertenece la opción (SOL_SOCKET para opciones genéricas). optname: nombre de la opción (SO_REUSEADDR). value: valor a asignar (1 para activar).
Valor retornado	No retorna valor (None).
Protocolo	Ambos.
Ejemplo de uso	<code>servidor_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)</code>
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 158.

3. bind()

Campo	Detalle
Propósito	Asocia (enlaza) el socket a una dirección IP y un número de puerto local específicos, para que el sistema operativo sepa a qué proceso entregar los datos entrantes por ese puerto.
Sintaxis (Python)	<code>socket.bind(address)</code>
Parámetros principales	address: tupla (host, puerto), por ejemplo ("127.0.0.1", 5050).
Valor retornado	No retorna valor (None). Lanza OSError si la dirección/puerto ya está en uso.
Protocolo	Ambos.
Ejemplo de uso	<code>servidor_socket.bind((HOST, PORT))</code>
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 165.

4. listen()

Campo	Detalle
Propósito	Habilita el socket para aceptar conexiones entrantes, transformándolo en un socket de tipo servidor y definiendo el tamaño máximo de la cola de conexiones pendientes.
Sintaxis (Python)	<code>socket.listen(backlog)</code>
Parámetros principales	backlog: número máximo de conexiones en espera de ser aceptadas antes de que el sistema operativo comience a rechazarlas.
Valor retornado	No retorna valor (None).
Protocolo	Exclusivo de TCP (solo aplica a sockets SOCK_STREAM).
Ejemplo de uso	<code>servidor_socket.listen(5)</code>
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 168.

5. accept()

Campo	Detalle
Propósito	Bloquea la ejecución hasta que llega una nueva conexión entrante, completando el three-way handshake de TCP, y crea un nuevo socket dedicado exclusivamente a la comunicación con ese cliente.
Sintaxis (Python)	<code>conn, addr = socket.accept()</code>
Parámetros principales	No recibe parámetros.

Campo	Detalle
Valor retornado	Una tupla (conn, addr): conn es el nuevo socket para comunicarse con el cliente, y addr es su dirección (IP, puerto).
Protocolo	Exclusivo de TCP.
Ejemplo de uso	cliente_socket, direccion = servidor_socket.accept()
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 176.

6. connect()

Campo	Detalle
Propósito	Inicia una conexión con un servidor remoto, ejecutando el three-way handshake de TCP (SYN, SYN-ACK, ACK) hasta establecer la conexión.
Sintaxis (Python)	socket.connect(address)
Parámetros principales	address: tupla (host, puerto) del servidor destino.
Valor retornado	No retorna valor (None). Lanza ConnectionRefusedError si no hay proceso escuchando en ese puerto, o socket.timeout si se agota el tiempo definido con settimeout().
Protocolo	Exclusivo de TCP (en UDP no existe una conexión persistente).
Ejemplo de uso	cliente_socket.connect((HOST, PORT))
¿Utilizado en el programa?	Sí
Referencia en el código	client.py, línea 34.

7. send()

Campo	Detalle
Propósito	Envía datos a través de un socket ya conectado hacia el otro extremo de la comunicación.
Sintaxis (Python)	socket.send(bytes)
Parámetros principales	bytes: los datos a enviar, codificados como una secuencia de bytes (por ejemplo, mediante .encode("utf-8")).
Valor retornado	Un entero que indica la cantidad de bytes efectivamente enviados.
Protocolo	Utilizado en sockets orientados a conexión (TCP); en UDP el equivalente es sendto().
Ejemplo de uso	cliente_socket.send(mensaje_limpio.encode("utf-8"))

Campo	Detalle
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, líneas 103 y 110; client.py, línea 91.

8. recv()

Campo	Detalle
Propósito	Recibe datos desde un socket conectado. La llamada se bloquea hasta que llegan datos o hasta que el otro extremo cierra la conexión.
Sintaxis (Python)	socket.recv(bufsize)
Parámetros principales	bufsize: número máximo de bytes a leer en la llamada (en esta aplicación, BUFFER_SIZE = 1024).
Valor retornado	Un objeto bytes con los datos recibidos. Retorna b"" cuando el otro extremo cerró la conexión de forma ordenada (segmento FIN).
Protocolo	Utilizado en sockets orientados a conexión (TCP); en UDP el equivalente es recvfrom().
Ejemplo de uso	datos = cliente_socket.recv(BUFFER_SIZE)
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, línea 86; client.py, línea 97.

9. settimeout()

Campo	Detalle
Propósito	Define un tiempo máximo (en segundos) que las operaciones bloqueantes del socket (connect(), recv(), send()) esperarán antes de lanzar una excepción socket.timeout.
Sintaxis (Python)	socket.settimeout(value)
Parámetros principales	value: tiempo en segundos (número flotante); None desactiva el timeout.
Valor retornado	No retorna valor (None).
Protocolo	Ambos.
Ejemplo de uso	cliente_socket.settimeout(15.0)
¿Utilizado en el programa?	Sí
Referencia en el código	client.py, línea 29.

10. close()

Campo	Detalle
Propósito	Cierra el socket y libera los recursos del sistema operativo asociados a él. En TCP, además, inicia el procedimiento de cierre de la conexión enviando un segmento FIN al otro extremo.
Sintaxis (Python)	socket.close()
Parámetros principales	No recibe parámetros.
Valor retornado	No retorna valor (None).
Protocolo	Ambos.
Ejemplo de uso	cliente_socket.close()
¿Utilizado en el programa?	Sí
Referencia en el código	server.py, líneas 140 y 211; client.py, línea 144.

6. Pruebas y resultados

Para validar el funcionamiento de la aplicación se ejecutaron 12 pruebas manuales, iniciando el servidor y el cliente en la misma máquina (dirección 127.0.0.1, puerto 5050). Las pruebas cubren tanto los casos de uso normales (inicio, conexión, mensajes, comandos, cierre ordenado) como los principales escenarios de error y seguridad (mensaje vacío, archivo inexistente, intento de Path Traversal, servidor no disponible, desconexión abrupta y puerto en uso).

#	Prueba	Estado
1	Inicio del servidor	Aprobada
2	Conexión del cliente	Aprobada
3	Mensaje normal	Aprobada
4	Mensaje vacío	Aprobada
5	Comando /hora	Aprobada
6	Comando /archivo (archivo válido)	Aprobada
7	Comando /archivo (archivo inexistente)	Aprobada
8	Comando /archivo con Path Traversal	Aprobada
9	Comando /salir (cierre ordenado)	Aprobada
10	Servidor no disponible	Aprobada
11	Cliente desconectado abruptamente	Aprobada
12	Puerto ya en uso	Aprobada

Prueba 1 — Inicio del servidor

Acción / comando	Resultado esperado
cd server && python3 server.py	El servidor queda escuchando en 127.0.0.1:5050

Se inició el servidor y se verificó en consola la aparición de los mensajes [INICIO] y [ESPERA], que confirman que el socket fue creado, asociado a la dirección y puerto, y que quedó habilitado para aceptar conexiones.

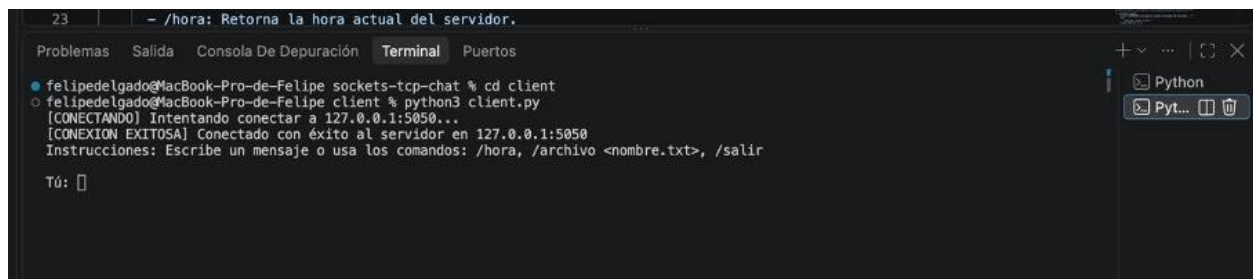


```
Problemas Salida Consola De Depuración Terminal Puertos
o felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
```

Prueba 2 — Conexión del cliente

Acción / comando	Resultado esperado
cd client && python3 client.py	Conexión TCP establecida correctamente entre ambos extremos

Con el servidor ya en ejecución, se inició el cliente. En la terminal del servidor se observó el mensaje [CONEXION] indicando la dirección del cliente conectado, y en la terminal del cliente se observó [CONEXION EXITOSA], confirmando el establecimiento de la conexión TCP en ambos extremos.



```
23 - /hora: Retorna la hora actual del servidor.
Problemas Salida Consola De Depuración Terminal Puertos
o felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd client
o felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[CONECTANDO] Intentando conectar a 127.0.0.1:5050...
[CONEXION EXITOSA] Conectado con éxito al servidor en 127.0.0.1:5050
Instrucciones: Escribe un mensaje o usa los comandos: /hora, /archivo <nombre.txt>, /salir
Tú: 
```

```
23 | - /hora: Retorna la hora actual del servidor.
Problemas Salida Consola De Depuración Terminal Puertos
○ felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
[CONEXION] Cliente conectado exitosamente desde ('127.0.0.1', 52768)
█
```

Prueba 3 — Mensaje normal

Acción / comando	Resultado esperado
Escribir: hola	El servidor responde con eco del mensaje recibido

Se envió el mensaje de texto libre "hola" desde el cliente. El servidor recibió el mensaje, lo procesó y respondió con la confirmación de recepción, visible en ambas terminales.

```
Problemas Salida Consola De Depuración Terminal Puertos
● felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd client
○ felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[CONECTANDO] Intentando conectar a 127.0.0.1:5050...
[CONEXION EXITOSA] Conectado con éxito al servidor en 127.0.0.1:5050
Instrucciones: Escribe un mensaje o usa los comandos: /hora, /archivo <nombre.txt>, /salir

Tú: hola
[SERVIDOR] Recibido: hola

Tú: █

23 | - /hora: Retorna la hora actual del servidor.
Problemas Salida Consola De Depuración Terminal Puertos
○ felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
[CONEXION] Cliente conectado exitosamente desde ('127.0.0.1', 52768)
[MENSAJE] ('127.0.0.1', 52768) -> hola
█
```

Prueba 4 — Mensaje vacío

Acción / comando	Resultado esperado
Presionar Enter sin escribir nada	El cliente rechaza el envío antes de tocar la red

Se intentó enviar un mensaje vacío presionando Enter sin ingresar texto. El cliente detectó la condición mediante la validación local y mostró una advertencia, sin llegar a enviar ningún dato por el socket.

```
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd client
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[CONEXION EXITOSA] Conectado con éxito al servidor en 127.0.0.1:5050
Instrucciones: Escribe un mensaje o usa los comandos: /hora, /archivo <nombre.txt>, /salir

Tú: hola
[SERVIDOR] Recibido: hola

Tú:
[ADVERTENCIA] No se permiten mensajes vacíos. Por favor ingresa texto.
Tú: 
```

Prueba 5 — Comando /hora

Acción / comando	Resultado esperado
/hora	El servidor responde con la fecha y hora actuales del sistema

Se envió el comando /hora. El servidor interpretó el comando dentro de procesar_mensaje() y respondió con la fecha y hora actuales obtenidas mediante el módulo datetime.

```
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
Instrucciones: Escribe un mensaje o usa los comandos: /hora, /archivo <nombre.txt>, /salir

Tú: hola
[SERVIDOR] Recibido: hola

Tú:
[ADVERTENCIA] No se permiten mensajes vacíos. Por favor ingresa texto.
Tú: /hora
[SERVIDOR] Hora actual: 2026-08-29 18:25:36

Tú: 
```

Prueba 6 — Comando /archivo (archivo válido)

Acción / comando	Resultado esperado
/archivo saludo.txt	El servidor devuelve el contenido del archivo solicitado

Se solicitó el archivo saludo.txt, presente en el directorio server/archivos/. El servidor validó el nombre, abrió el archivo y devolvió su contenido completo al cliente.

```
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
Tú:
[ADVERTENCIA] No se permiten mensajes vacíos. Por favor ingresa texto.
Tú: /hora
[SERVIDOR] Hora actual: 2026-08-29 18:25:36

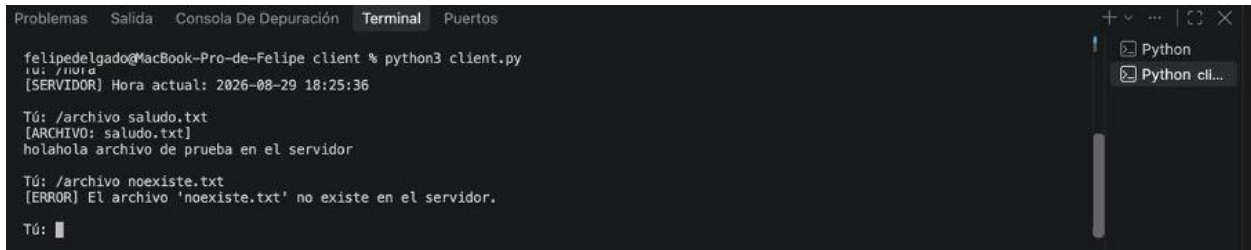
Tú: /archivo saludo.txt
[ARCHIVO: saludo.txt]
holahola archivo de prueba en el servidor

Tú: 
```

Prueba 7 — Comando /archivo (archivo inexistente)

Acción / comando	Resultado esperado
/archivo noexiste.txt	Mensaje de error controlado, sin caída del servidor

Se solicitó un archivo que no existe en el servidor. La excepción FileNotFoundError fue capturada correctamente y se devolvió un mensaje de error descriptivo al cliente, sin que el servidor se detuviera ni afectara a otras conexiones.



```
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[SERVIDOR] Hora actual: 2026-08-29 18:25:36

Tú: /archivo saludo.txt
[ARCHIVO: saludo.txt]
holahola archivo de prueba en el servidor

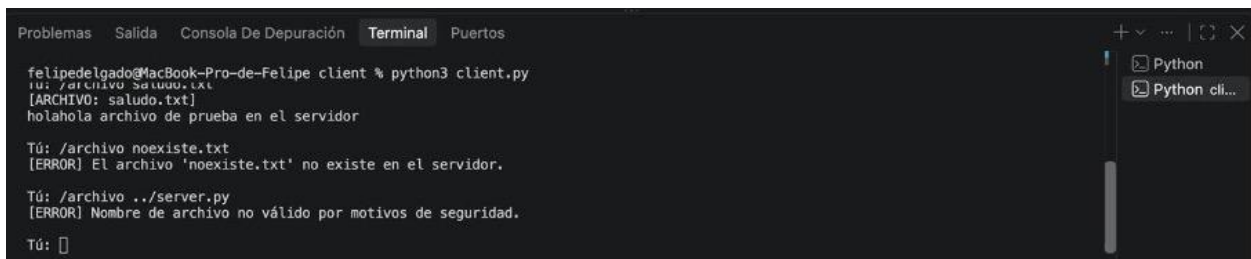
Tú: /archivo noexiste.txt
[ERROR] El archivo 'noexiste.txt' no existe en el servidor.

Tú: █
```

Prueba 8 — Comando /archivo con Path Traversal

Acción / comando	Resultado esperado
/archivo ../server.py	El servidor bloquea el acceso fuera del directorio permitido

Se intentó acceder a un archivo fuera del directorio server/archivos/ mediante una ruta relativa (../server.py). La validación de seguridad, que compara el nombre base de la ruta resuelta contra el nombre recibido, detectó el intento y lo bloqueó, devolviendo un mensaje de error sin exponer el archivo del servidor.



```
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[SERVIDOR] Hora actual: 2026-08-29 18:25:36

Tú: /archivo saludo.txt
[ARCHIVO: saludo.txt]
holahola archivo de prueba en el servidor

Tú: /archivo noexiste.txt
[ERROR] El archivo 'noexiste.txt' no existe en el servidor.

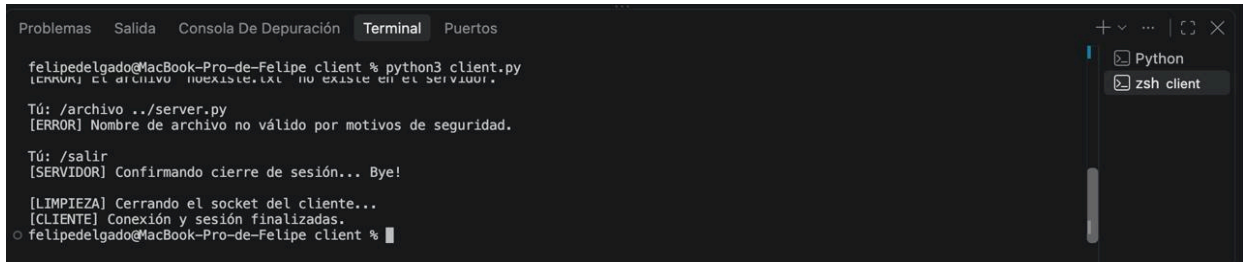
Tú: /archivo ../server.py
[ERROR] Nombre de archivo no válido por motivos de seguridad.

Tú: █
```

Prueba 9 — Comando /salir (cierre ordenado)

Acción / comando	Resultado esperado
/salir	El servidor confirma el cierre y ambos extremos cierran la sesión de forma ordenada

Se envió el comando /salir. El servidor respondió con un mensaje de confirmación de cierre, y tanto el cliente como el socket del servidor asociado a esa sesión se cerraron de forma ordenada mediante close().



```
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[ERROR] El archivo no existe.txt no existe en el servidor.

Tú: /archivo ../server.py
[ERROR] Nombre de archivo no válido por motivos de seguridad.

Tú: /salir
[SERVIDOR] Confirmando cierre de sesión... Bye!

[LIMPIEZA] Cerrando el socket del cliente...
[CLIENTE] Conexión y sesión finalizadas.
o felipedelgado@MacBook-Pro-de-Felipe client %
```



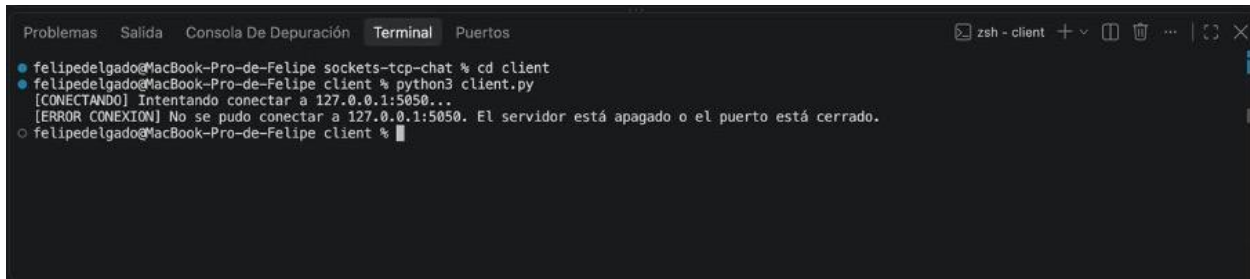
```
o felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
[CONEXION] Cliente conectado exitosamente desde ('127.0.0.1', 54260)
[MENSAJE] ('127.0.0.1', 54260) -> /salir
[INFO] Cliente ('127.0.0.1', 54260) solicitó desconexión mediante /salir.
[DESCONEXION] Socket y recursos del cliente ('127.0.0.1', 54260) liberados de forma segura.

[ESPERA] Esperando conexión entrante de algún cliente...
█
```

Prueba 10 — Servidor no disponible

Acción / comando	Resultado esperado
Ejecutar el cliente sin un servidor activo escuchando	El cliente informa el error de conexión de forma controlada

Se detuvo el servidor (o se ejecutó un cliente nuevo sin servidor activo) y se intentó conectar. El cliente capturó la excepción `ConnectionRefusedError` e informó por consola que el servidor está apagado o el puerto está cerrado, finalizando de forma controlada sin generar una excepción no manejada.

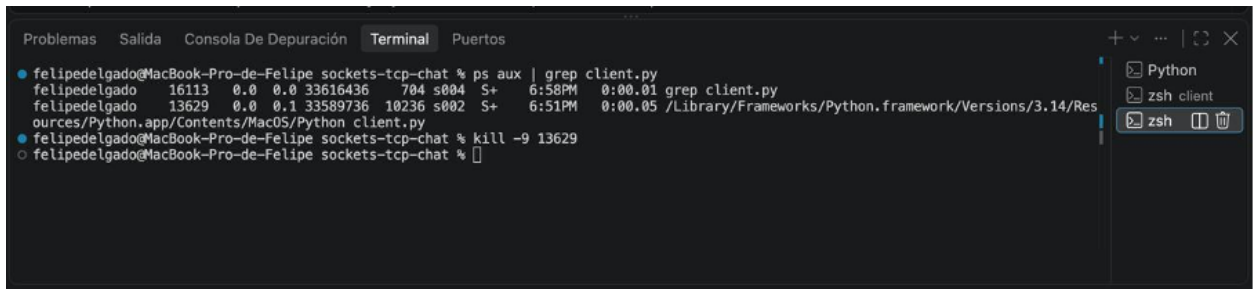


```
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd client
felipedelgado@MacBook-Pro-de-Felipe client % python3 client.py
[CONECTANDO] Intentando conectar a 127.0.0.1:5050...
[ERROR CONEXION] No se pudo conectar a 127.0.0.1:5050. El servidor está apagado o el puerto está cerrado.
o felipedelgado@MacBook-Pro-de-Felipe client %
```

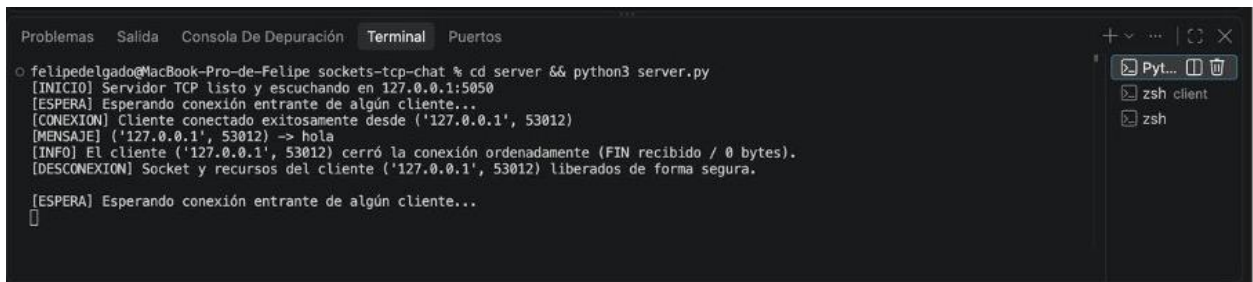
Prueba 11 — Cliente desconectado abruptamente

Acción / comando	Resultado esperado
Cerrar la ventana del cliente a la fuerza (o kill) sin enviar /salir	El servidor detecta la desconexión y vuelve a esperar nuevas conexiones

Con ambos procesos en ejecución, se cerró el cliente de forma abrupta (sin enviar `/salir`). El servidor detectó la pérdida de la conexión, liberó el socket del cliente en el bloque `finally` y volvió al estado de espera, mostrando nuevamente el mensaje `[ESPERA]` sin detener su ejecución.



```
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % ps aux | grep client.py
felipedelgado  16113  0.0  0.0  33616436   704 s004  S+   6:58PM   0:00.01  grep client.py
felipedelgado  13629  0.0  0.1  33589736  10236 s002  S+   6:51PM   0:00.05  /Library/Frameworks/Python.framework/Versions/3.14/Resources/Python.app/Contents/MacOS/Python client.py
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % kill -9 13629
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat %
```

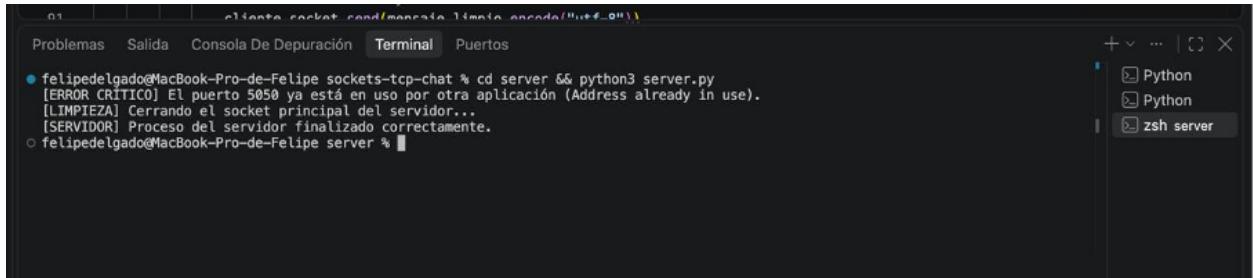


```
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
[CONEXION] Cliente conectado exitosamente desde ('127.0.0.1', 53012)
[MENSAJE] ('127.0.0.1', 53012) -> hola
[INFO] El cliente ('127.0.0.1', 53012) cerró la conexión ordenadamente (FIN recibido / 0 bytes).
[DESCONEXION] Socket y recursos del cliente ('127.0.0.1', 53012) liberados de forma segura.
[ESPERA] Esperando conexión entrante de algún cliente...
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat %
```

Prueba 12 — Puerto ya en uso

Acción / comando	Resultado esperado
Con el servidor corriendo, ejecutar python3 server.py en otra terminal	El segundo proceso informa el error y finaliza limpiamente

Con el servidor original ya en ejecución sobre el puerto 5050, se intentó levantar una segunda instancia del servidor en el mismo puerto. El sistema operativo rechazó el bind() con un error de tipo OSError (dirección en uso), el cual fue capturado explícitamente, informado por consola, y el segundo proceso finalizó limpiamente sin afectar al servidor original.



```
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[INICIO] Servidor TCP listo y escuchando en 127.0.0.1:5050
[ESPERA] Esperando conexión entrante de algún cliente...
[CONEXION] Cliente conectado exitosamente desde ('127.0.0.1', 53012)
[MENSAJE] ('127.0.0.1', 53012) -> hola
[INFO] El cliente ('127.0.0.1', 53012) cerró la conexión ordenadamente (FIN recibido / 0 bytes).
[DESCONEXION] Socket y recursos del cliente ('127.0.0.1', 53012) liberados de forma segura.
[ESPERA] Esperando conexión entrante de algún cliente...
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat % cd server && python3 server.py
[ERROR CRITICO] El puerto 5050 ya está en uso por otra aplicación (Address already in use).
[LIMPIEZA] Cerrando el socket principal del servidor...
[SERVIDOR] Proceso del servidor finalizado correctamente.
felipedelgado@MacBook-Pro-de-Felipe sockets-tcp-chat %
```

7. Conclusiones

7.1 Principales aprendizajes

El desarrollo de esta actividad permitió comprender en la práctica cómo se construye una comunicación de red utilizando directamente la API de sockets, sin la abstracción adicional de frameworks de alto nivel. Se aprendió a distinguir con claridad el rol de cada método dentro del ciclo de vida de una conexión TCP

(creación, asociación, escucha, aceptación, envío, recepción y cierre), así como la importancia de manejar de forma explícita los distintos tipos de error que pueden ocurrir en una comunicación real.

7.2 Resultados obtenidos

Se logró implementar una aplicación de chat cliente-servidor completamente funcional sobre TCP, que cumple con todos los requisitos mínimos establecidos en la pauta: componente cliente y servidor, intercambio efectivo de datos, configuración explícita de IP y puerto, validación de datos recibidos, manejo de errores, cierre correcto de la comunicación y una funcionalidad reconocible (sistema de comandos) más allá de la simple conexión. Las 12 pruebas realizadas confirmaron el correcto funcionamiento tanto de los casos de uso normales como de los distintos escenarios de error.

7.3 Dificultades encontradas

Uno de los principales desafíos fue diferenciar correctamente los distintos escenarios de cierre y error de una conexión TCP (cierre ordenado mediante FIN versus desconexión abrupta mediante RST), y capturar cada excepción de forma específica en lugar de utilizar únicamente un bloque genérico. Asimismo, implementar la protección contra Path Traversal en el comando /archivo exigió comprender cómo Python resuelve rutas de archivos para evitar accesos no autorizados fuera del directorio permitido.

7.4 Diferencias observadas entre TCP y UDP

A partir del trabajo con TCP quedó en evidencia el costo y beneficio de trabajar con un protocolo orientado a la conexión: por un lado, TCP simplificó el diseño de la aplicación al garantizar que los mensajes llegaran completos y en orden, sin necesidad de implementar manualmente confirmaciones o reintentos; por otro lado, esto implica una mayor sobrecarga inicial (el handshake de conexión) y una gestión más compleja del ciclo de vida de la conexión, en comparación con el modelo sin conexión de UDP, donde el desarrollador debe encargarse manualmente de cualquier mecanismo de fiabilidad que la aplicación requiera.

7.5 Posibles mejoras y funcionalidades futuras

- Convertir el servidor en un servidor concurrente para atender a múltiples clientes de forma simultánea.
- Incorporar cifrado de la comunicación (por ejemplo, mediante TLS/SSL) para proteger la confidencialidad de los mensajes.
- Extender el sistema de comandos para permitir subir archivos desde el cliente hacia el servidor, no solo descargarlos.

8. Anexos

8.1 Código fuente completo

<https://github.com/FelipeDelgado81/sockets-tcp-chat.git>

8.2 Instrucciones de ejecución

Ver el archivo README.md incluido en la entrega, el cual detalla los requisitos de instalación, la versión de Python utilizada, las bibliotecas necesarias, los comandos para iniciar el servidor y el cliente, la dirección IP y puerto utilizados, y un ejemplo de ejecución.

8.3 Estructura de archivos entregados

```
sockets-tcp-chat/  
├── server/  
│   ├── server.py  
│   └── archivos/  
│       └── saludo.txt  
├── client/  
│   └── client.py  
├── README.md  
└── Informe.pdf
```